

MacHuna SWS Audio Format — Technical Specification

Overview

Audio support in the Kahuna SWS format is implemented by appending raw PCM audio data after the key video plane, with additional fields populated in the 512-byte header. This spec was reverse-engineered from a real K-Watch-generated SWS file (`TESTAUDIO1.mov`), 6 frames, 1920x1080, 25fps).

SWS File Layout (with audio)

[0x000 - 0x1FF]	512-byte header
[0x200 - N]	Fill plane (plane_size × frame_count bytes)
[N - M]	Key plane (plane_size × frame_count bytes)
[M - EOF]	Audio data (5,760 × frame_count × 16 channels bytes)

Header Fields (audio-specific)

All values big-endian unless noted.

Offset	Size	Value	Description
0x1C2	uint16	0x1680 = 5,760	Audio bytes per frame per channel. At 48kHz/25fps/24-bit: 48000÷25×3 = 5,760
0x1CC	uint32	total file size	Same as non-audio but now includes audio data
0x1E8	uint32	audio_offset ÷ 32	Pointer to start of audio data. audio_offset = 512 + plane_size × frame_count × 2
0x1EC	uint32	0x03000000	Audio format flag. Exact meaning unknown but always this value in observed file

Fields unchanged from non-audio SWS

All other header fields remain identical. See main MacHuna codebase for full header spec.

Audio Data Format

- **Sample rate:** 48,000 Hz

- **Bit depth:** 24-bit signed PCM (little-endian, 3 bytes per sample)
- **Channels:** 16 (8 stereo pairs — channel pairs 1-2, 3-4, 5-6, 7-8, 9-10, 11-12, 13-14, 15-16)
- **Layout:** All 16 channels for frame 1, then all 16 channels for frame 2, etc.
- **Samples per frame:** 1,920 ($48,000 \div 25\text{fps}$)
- **Bytes per frame (all channels):** $5,760 \times 16 = 92,160$ bytes
- **Total audio size:** $92,160 \times \text{frame_count}$ bytes

Channel pair mapping (from K-Watch manual)

- Audio group 1 = channels 1 & 2
 - Audio group 2 = channels 3 & 4
 - Audio group 3 = channels 5 & 6
 - Audio group 4 = channels 7 & 8
 - Groups 5-8 = channels 9-16 (extension, less commonly used)
-

Implementation Plan for MacHuna

Step 1 — Extract audio from MOV

Use ffmpeg to extract audio as raw 24-bit PCM, 48kHz, remixed to 16 channels:

```
bash

ffmpeg -i input.mov \
  -vn \
  -acodec pcm_s24le \
  -ar 48000 \
  -ac 16 \
  -f s24le \
  audio_raw.pcm
```

If source has fewer than 16 channels, ffmpeg will pad with silence automatically using `-ac 16`.

Step 2 — Verify audio frame alignment

Audio must be exactly `frame_count × 1920 samples × 16 channels × 3 bytes` in total. Pad with zero bytes if necessary to align.

Step 3 — Append audio after key plane

Simply concatenate: header + fill + key + audio_pcm

Step 4 — Update header fields

```
python

audio_offset = SWS_HEADER_SIZE + plane_size * frame_count * 2
audio_frame_size = 5760 # 48000/25 * 3

struct.pack_into('>H', hdr, 0x1C2, audio_frame_size)
struct.pack_into('>I', hdr, 0x1E8, audio_offset // 32)
struct.pack_into('>I', hdr, 0x1EC, 0x03000000)
# 0x1CC total_size updated last to include audio
total_size = audio_offset + (audio_frame_size * 16 * frame_count)
struct.pack_into('>I', hdr, 0x1CC, total_size)
```

Step 5 — GUI changes

- Add "Include Audio" checkbox to File Settings (default: on)
 - If source has no audio track, skip audio extraction silently
 - Existing "Exclude Audio" logic already in place -- just wire it to skip Steps 1-4
-

Notes and Unknowns

- `0x1EC` = `0x03000000` -- exact meaning not confirmed. Possibly encodes bit depth (24-bit = 3) or channel count. Always use this value until a different configuration is tested.
 - For 50fps content (1080p50), samples per frame = $48000 \div 50 = 960$, so `audio_frame_size` = $960 \times 3 = 2,880$. `0x1C2` would need updating accordingly.
 - TGA sequence audio (WAV files alongside TGA sequences) is explicitly out of scope for MacHuna v1.x.
 - Split files (>4GB): audio handling for multi-chunk SWS not yet investigated.
-

Reference File

- Source: `TESTAUDIO1.mov`, 6 frames, 1920×1080, 25fps, Rec.709
- Generated by: K-Watch (Windows), K-Manager Pro v9.6 Release 1
- Audio excluded file size: 66,355,712 bytes
- Audio included file size: 66,540,032 bytes
- Audio data size: 184,320 bytes = $5,760 \times 6 \text{ frames} \times 16 \text{ channels}$ (confirms format)